

Algoritmos de Engenharia

Estruturas de Dados—Introdução (Parte III)

Dictionary

Algorithms may require several types of operations to be performed on sets. For example, many algorithms need only the ability to insert elements into, delete elements from, and test membership in a set. We call a dynamic set that supports these operations a *dictionary*. Other algorithms require more complicated operations. For example, min-priority queues, which Chapter 6 introduced in the context of the heap data structure, support the operations of inserting an element into and extracting the smallest element from a set. The best way to implement a dynamic set depends upon the operations that you need to support.

Typical operations

Search(S, k)

A query that, given a set S and a key value k , returns a pointer x to an element in S such that $x.key = k$, or NIL if no such element belongs to S .

Insert(S, x)

A modifying operation that adds the element pointed to by x to the set S . We usually assume that any attributes in element x needed by the set implementation have already been initialized.

Delete(S, x)

A modifying operation that, given a pointer x to an element in the set S , removes x from S . (Note that this operation takes a pointer to an element x , not a key value.)

Minimum(S) and Maximum(S)

Queries on a totally ordered set S that return a pointer to the element of S with the smallest (for Minimum) or largest (for Maximum) key.

Successor(S, x)

A query that, given an element x whose key is from a totally ordered set S , returns a pointer to the next larger element in S , or NIL if x is the maximum element.

Predecessor(S, x)

A query that, given an element x whose key is from a totally ordered set S , returns a pointer to the next smaller element in S , or NIL if x is the minimum element.

Arrays

Arrays store elements contiguously in memory.

- If
 - the first element of an array has index s ,
 - the array starts at memory address a , and
 - each element occupies b bytes,

then the i th element occupies bytes $a + b(i - s)$ through $a + b(i + 1 - s) - 1$.

The most common values for s are 0 and 1.

- $s = 0 \Rightarrow a + bi$ through $a + b(i + 1) - 1$.
- $s = 1 \Rightarrow a + b(i - 1)$ through $a + bi - 1$.

Arrays

- The computer can access any array element in constant time (assuming that the computer can access all memory locations in same amount of time).
- If elements of an array occupy different numbers of bytes, elements might be accessed incorrectly or not in constant time. Therefore, most programming languages require that each element of an array must be the same size. Sometimes, pointers to objects are stored instead of objects themselves in order to meet this requirement.

Matrices

Notation: an $m \times n$ matrix has m rows and n columns.

We represent a matrix with one or more arrays.

Two common ways to store a matrix:

- **Row-major**: matrix is stored row by row.
- **Column-major**: matrix is stored column by column.

Example: Consider the 2×3 matrix

$$M = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{pmatrix}.$$

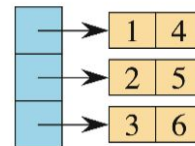
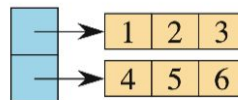
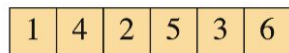
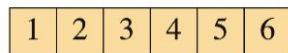
Row-major order: store two rows 1 2 3 and 4 5 6

Column-major order: store three columns 1 4; 2 5; and 3 6.

Matrices

There are many ways to store M . Shown below, from left to right, are four possible ways:

1. In row-major order, single array.
2. In column-major order, single array.
3. In row-major order, one array per row with a single array of pointers to the row arrays.
4. In column-major order, one array per column with a single array of pointers to the column arrays.



There are other ways to store matrices.

Stacks and Queues

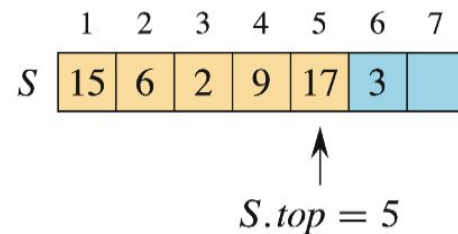
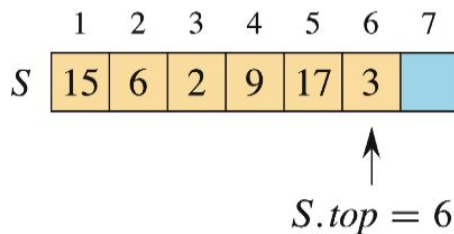
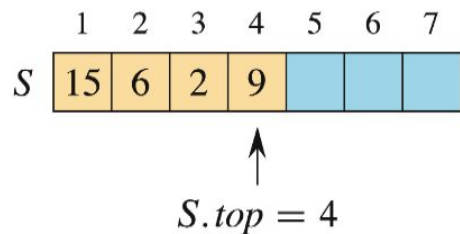
Stacks and queues are dynamic sets in which the element removed from the set by the DELETE operation is prespecified.

Stack: the element deleted is the one that was most recently inserted. Stacks use a *last-in, first-out*, or ***LIFO***, policy.

Queue: the element deleted is the one that has been in the set for the longest time. Queues use a *first-in, first-out*, or ***FIFO***, policy.

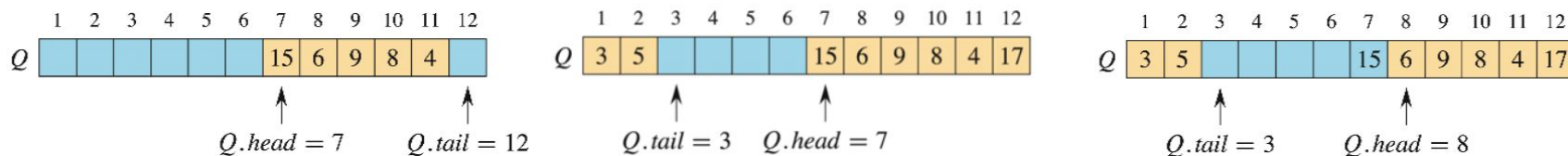
Stacks

Implement a stack of at most n elements with an array $S[1:n]$. Attribute $S.top$ indexes the most recently inserted element. The stack contains elements $S[1:S.top]$, where $S[1]$ is the element at the bottom of the stack and $S[S.top]$ is the element at the top. Attribute $S.size = n$ gives the size of the array.



Queues

Implement a queue of at most $n - 1$ elements with an array $Q[1:n]$.



- Initially, $Q.head = Q.tail = 1$.
- $Q.head = Q.tail \Rightarrow$ queue is empty. Attempting to dequeue causes underflow.
- $Q.head = Q.tail + 1$ or both $Q.head = 1$ and $Q.tail = n \Rightarrow$ the queue is full. Attempting to enqueue causes overflow.
- Attribute $Q.size$ gives the size n of the array.

Linked Lists

A *linked list* has

- Objects arranged in a linear order.
- Order is determined by a pointer in each object.

In a *doubly linked list*, each element x has the following attributes:

- $x.key$
- $x.next$: the successor of x , NIL if x has no successor so that it's the *tail*
- $x.prev$: the predecessor of x , NIL if x has no predecessor so that it's the *head*

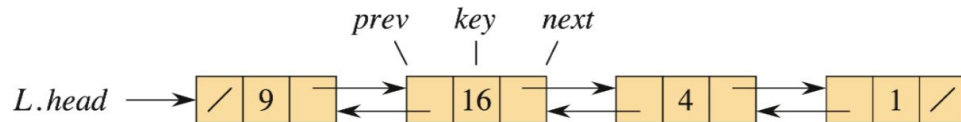
$L.head$ points to the first element of the list, NIL if the list is empty.

Linked Lists

Linked lists come in several types:

- ***Singly linked***: each element has a next attribute but not a *prev* attribute.
- ***Sorted***: the linear order of the list follows the linear order of keys stored in elements of the list.
- ***Unsorted***: the elements can appear in any order.
- ***Circular***: the *prev* pointer of the head of the list points to the tail, and the next pointer of the tail of the list points to the head.

Here is a doubly linked list whose elements have keys 9, 16, 4, 1. Slashes indicate NIL.

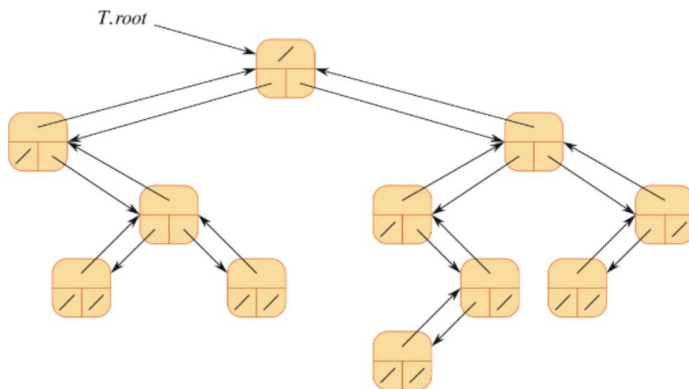


Binary Trees

Give each node in a binary tree the following attributes: p (parent), $left$, $right$.

If $x.p = \text{NIL}$, x is the root. The root of tree T is $T.\text{root}$. If $T.\text{root} = \text{NIL}$, then T is empty.

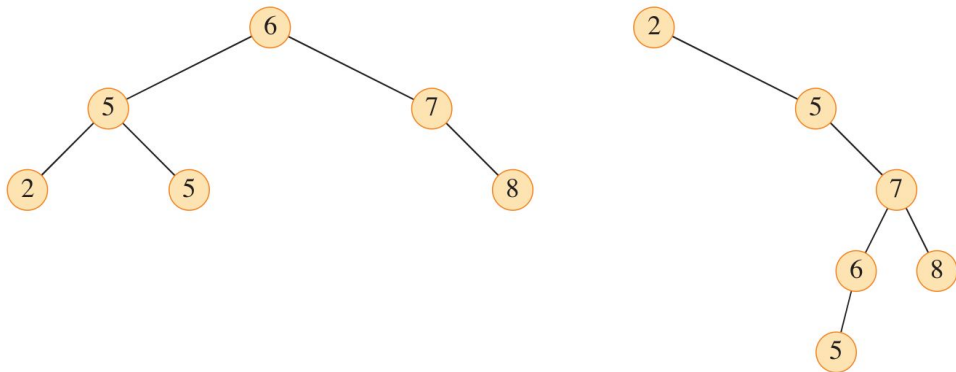
If x has no left child, then $x.left = \text{NIL}$. Same for right child.



Binary Search Trees

The keys in a binary search tree are always stored in such a way as to satisfy the *binary-search-tree property*:

Let x be a node in a binary search tree. If y is a node in the left subtree of x , then $y.key \leq x.key$. If y is a node in the right subtree of x , then $y.key \geq x.key$.



Balanced binary search trees: the height of the tree is $O(\log n)$.

Exercises

1. Implemente as estruturas de dados: 1) array ordenado, 2) lista encadeada não ordenada, e 3) árvore de busca binária balanceada (e.g., AVL, Red-Black trees). Em seguida, insira n elementos em cada estrutura de dados. Depois disso, considerando diferentes valores de n , compare os tempos de *inserção* e *busca* de cada uma.

Hash Tables

Many applications require a dynamic set that supports only the *dictionary operations* INSERT, SEARCH, and DELETE. Example: a symbol table in a compiler.

A hash table is effective for implementing a dictionary.

- The expected time to search for an element in a hash table is $O(1)$, under some reasonable assumptions.
- Worst-case search time is $\Theta(n)$, however.

Hash Tables

A hash table is a generalization of an ordinary array.

- With an ordinary array, store the element whose key is k in position k of the array.
- Given a key k , to find the element whose key is k , just look in the k th position of the array. This is called *direct addressing*.
- Direct addressing is applicable when you can afford to allocate an array with one position for every possible key.

Hash Tables

Use a hash table when do not want to (or cannot) allocate an array with one position per possible key.

- Use a hash table when the number of keys actually stored is small relative to the number of possible keys.
- A hash table is an array, but it typically uses a size proportional to the number of keys to be stored (rather than the number of possible keys).
- Given a key k , don't just use k as the index into the array.
- Instead, compute a function of k , and use that value to index into the array. We call this function a *hash function*.

[Python's Dictionaries are Hash Tables](#)

Hash Tables

Idea

Instead of storing an element with key k in slot k , use a function h and store the element in slot $h(k)$.

- We call h a ***hash function*** and T a ***hash table***.
- $h : U \rightarrow \{0, 1, \dots, m - 1\}$, so that $h(k)$ is a legal slot number in T .
- We say that k ***hashes*** to slot $h(k)$.

Hash Tables

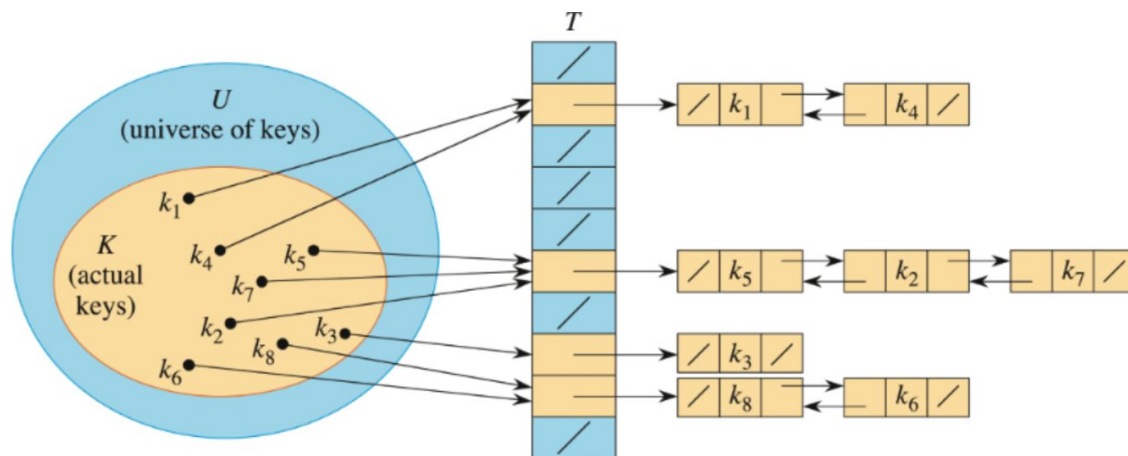
Collision

When two or more keys hash to the same slot.

- Can happen when there are more possible keys than slots ($|U| > m$).
- For a given set K of keys with $|K| \leq m$, may or may not happen. Definitely happens if $|K| > m$.
- Therefore, must be prepared to handle collisions in all cases.
- Use two methods: chaining and open addressing. We'll examine both.

Hash Tables: Chaining

Like a nonrecursive type of divide-and-conquer: use the hash function to divide the n elements randomly into m subsets, each with approximately $|n/m|$ elements. Manage each subset independently as a linked list.



This figure shows singly linked lists. If needed to delete elements, it's better to use doubly linked lists.

Hash Tables: Open Addressing

Idea

- Store all elements in the hash table itself.
- Each slot contains either an element or NIL.
- The hash table can fill up, but the load factor can never be > 1 .
- How to handle collisions during insertion:
 - Determine the element's "first-choice" location in the hash table.
 - If the first-choice location is unoccupied, put the element there.
 - Otherwise, determine the element's "second-choice" location. If unoccupied, put the element there.
 - Otherwise, try the "third-choice" location. And so on, until an unoccupied location is found.
 - Different elements have different preference orders.

Exercises

2. Implemente tabelas *hash* usando funções *hash* com encadeamento e usando endereçamento aberto. Realize experimentos para mostrar numericamente as vantagens e desvantagens de cada caso, considerando as operações de inserção, busca e remoção.

Hash Tables: Computing Probe Sequences

The ideal situation is *independent uniform permutation hashing* (also known as *uniform hashing*): each key is equally likely to have any of the $m!$ permutations of $\langle 0, 1, \dots, m - 1 \rangle$ as its probe sequence. (This generalizes independent uniform hashing for a hash function that produces a whole probe sequence rather than just a single number.)

It's hard to implement true independent uniform permutation hashing. Instead, approximate it.

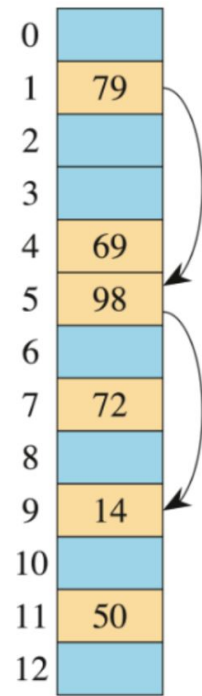
Hash Tables: Double Hashing

Uses auxiliary hash functions h_1, h_2 and probe number i :

$$h(k, i) = (h_1(k) + i h_2(k)) \bmod m .$$

The first probe goes to slot $h_1(k)$ (because i starts at 0). Successive probes are offset from previous slots by $h_2(k)$ modulo $m \Rightarrow$ the probe sequence depends on the key in two ways.

Example: $m = 13$, $h_1(k) = k \bmod 13$, $h_2(k) = 1 + (k \bmod 11)$, inserting key $k = 14$. First probe is to slot 1 ($14 \bmod 13 = 1$), which is occupied. Second probe is to slot 5 ($((14 \bmod 13) + (1 + (14 \bmod 11))) \bmod 13 = (1 + 4) \bmod 13 = 5$), which is occupied. Third probe is to slot 9 (offset from slot 5 by 4), which is free, so key 14 goes there.



Hash Tables: Linear Probing

Special case of double hashing.

Given auxiliary hash function h' , the probe sequence starts at slot $h'(k)$ and continues sequentially through the table, wrapping after slot $m - 1$ to slot 0.

Given key k and probe number i ($0 \leq i < m$), $h(k, i) = (h'(k) + i) \bmod m$.

The initial probe determines the entire sequence $\Rightarrow$ only m possible sequences.

Exercises

2. Implemente tabelas *hash* com encadeamento e usando endereçamento aberto. Realize experimentos para mostrar numericamente as vantagens e desvantagens de cada caso, considerando as operações de inserção, busca e remoção.
3. Compare implementações de tabelas *hash* com *hashing* duplo e *probing* linear. Analise e justifique os resultados dos seus experimentos.

Exercises

1. Implemente as estruturas de dados: 1) array ordenado, 2) lista encadeada não ordenada, e 3) árvore de busca binária balanceada (e.g., AVL, Red-Black trees). Em seguida, insira n elementos em cada estrutura de dados. Depois disso, considerando diferentes valores de n , compare os tempos de *inserção* e *busca* de cada uma.
2. Implemente tabelas *hash* com encadeamento e usando endereçamento aberto. Realize experimentos para mostrar numericamente as vantagens e desvantagens de cada caso, considerando as operações de inserção, busca e remoção.
3. Compare implementações de tabelas *hash* com *hashing* duplo e *probing* linear. Analise e justifique os resultados dos seus experimentos.